React Hook Form (RHF) ร่วมกับ **Axios** และการจัดการสถานะที่ถูกต้อง

Function 6.1 - The Secure Feedback Collector

---

1. จุดประสงค์

1. เพื่อฝึกการใช้ React Hook Form ในการจัดการสถานะของฟอร์มแบบ Uncontrolled
2. เพื่อฝึกการเขียน Validation Logic ที่ซับซ้อนด้วย Regex และ Custom Rules
3. เพื่อทำความเข้าใจการเชื่อมต่อ Frontend เข้ากับ REST API และการจัดการ Response/Error

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

---

2. โจทย์โดยละเอียด

พัฒนาหน้า **"Customer Feedback Form"** โดยมีเงื่อนไขดังนี้:

- **ฟิลด์ที่ต้องการ:**
  - `fullName` : (Required) ต้องเป็นตัวอักษรภาษาอังกฤษหรือไทยเท่านั้น ห้ามมีตัวเลข
  - `email` : (Required) ต้องเป็นรูปแบบ Email ที่ถูกต้อง
  - `category` : (Select) เลือกประเภท (Bug, Suggestion, Inquiry)
  - `message` : (Required) ความยาว **20 - 500 ตัวอักษร**
- **การทำงาน:**
  - ห้ามส่งข้อมูลหากยังติด Error (ใช้ `handleSubmit` ของ RHF)
  - ระหว่างส่งข้อมูล ปุ่ม Submit ต้องเปลี่ยนเป็นสถานะ **Disabled** และแสดง Loading
  - เมื่อส่งสำเร็จ ต้องเคลียร์ฟอร์ม และแสดงข้อมูลที่ Server ตอบกลับมา (JSON) ให้ผู้ใช้เห็นเพื่อยืนยัน
  - หาก Server ตอบกลับเป็น Error (เช่น 400 หรือ 500) ต้องแสดงแจ้งเตือนที่เหมาะสม

3. ขั้นตอนการทำ

1. **Setup:** ติดตั้ง Library ที่จำเป็น: `npm install react-hook-form axios react-hot-toast`
2. **Implementation:** เติมส่วนที่ `TODO` ในโค้ดให้สมบูรณ์ โดยอ้างอิงจากเงื่อนไขที่โจทย์กำหนด
3. **UI/UX:** ตกแต่ง CSS (แนะนำ Tailwind) ให้ฟิลด์ที่เกิด Error มีขอบสีแดง และแสดงข้อความ Error ใต้ฟิลด์นั้นๆ
4. **Testing:**
   - ลองกด Submit โดยไม่กรอกข้อมูล (ต้องติด Error)
   - ลองกรอก Message แค่ 5 ตัวอักษร (ต้องติด Error)
   - ลองกรอกข้อมูลให้ครบและถูกต้อง (ต้องแสดง Success Toast และโชว์ JSON)

4. คำถาม

1. ทำไมเราถึงใช้ `isSubmitting` มาดักปุ่ม `disabled` ? หากไม่ทำจะเกิดผลเสียอย่างไรต่อระบบ Full-stack?
2. หากต้องการเพิ่มฟิลด์ "เบอร์โทรศัพท์" ที่รับเฉพาะเลข 10 หลัก นิสิตจะเขียน `pattern` ใน `register` อย่างไร?

Checklist Function 6.1

C1: Validation Logic (ความถูกต้องของข้อมูล)

วิธีดูว่าผ่าน:

- [ ] **Full Name:** ลองพิมพ์ตัวเลข (เช่น `John 123`) หรืออักขระพิเศษลงไป ระบบต้อง **ไม่อนุญาต** และแสดงข้อความเตือนทันที
- [ ] **Email:** ลองพิมพ์อีเมลที่ไม่มี `@` หรือไม่มี `.com` (เช่น `test@mail`) ระบบต้องดักจับได้ว่าผิด Format
- [ ] **Message Length:** ลองพิมพ์ข้อความสั้นๆ (เช่น `Hello`) แล้วกดส่ง ระบบต้องแสดง Error แจ้งว่าต้องมีอย่างน้อย **20 ตัวอักษร**
- [ ] **Required Fields:** ลองปล่อยว่างทุกช่องแล้วกด Submit ระบบต้องแสดง Error Message ในทุกช่องที่จำเป็น

C2: React Hook Form Implementation (การใช้เครื่องมือ)

วิธีดูว่าผ่าน:

- [ ] **No Page Refresh:** เมื่อกดปุ่ม Submit หน้าเว็บต้อง **ไม่รีโหลด** (เพราะใช้ `handleSubmit` ที่จัดการ `e.preventDefault()` ให้แล้ว)
- [ ] **Registration:** ทุก Input ต้องถูกผูกด้วย `{...register('fieldName', {...})}` ไม่มีการใช้ `value` หรือ `onChange` แบบ Manual (ซึ่งจะทำให้ Re-render บ่อย)
- [ ] **Error Object:** ข้อความแจ้งเตือนต้องดึงมาจาก `errors.fieldName.message` เท่านั้น ไม่ใช่การ Hardcode ข้อความค้างไว้บนหน้าจอ

C3: Axios & API Communication (การเชื่อมต่อ)

วิธีดูว่าผ่าน:

- [ ] **POST Request:** เปิดหน้า **Inspect > Network** ใน Browser แล้วดูว่าเมื่อกดส่ง มีรายการ `posts` วิ่งออกไปเป็นแบบ **Method: POST** หรือไม่
- [ ] **Payload:** ตรวจดูใน Network Tab ว่าข้อมูลที่ส่งไป (Request Payload) มีโครงสร้างเป็น JSON ที่ถูกต้องตามที่เรากรอกหรือไม่
- [ ] **Handling Response:** เมื่อ API ตอบกลับมา (Status 201 Created) ข้อมูลนั้นต้องถูกนำมาแสดงในส่วน **Server Response** ด้านล่างฟอร์มจริง

C4: UX & Visual Feedback (ประสบการณ์ผู้ใช้)

วิธีดูว่าผ่าน:

- [ ] **Button State:** ขณะที่รอ API ตอบกลับ (Loading) ปุ่มต้องเปลี่ยนเป็นสีเทา หรือคลิกซ้ำไม่ได้ (ป้องกัน **Double Submit**)
- [ ] **Visual Error:** ช่องที่กรอกผิดควรมี **Border สีแดง** หรือสัญลักษณ์ที่บ่งบอกว่าช่องนั้นมีปัญหา (ไม่ใช่แค่โชว์ข้อความ)
- [ ] **Toast Notification:** ต้องมี Pop-up เด้งขึ้นมาบอกสถานะ (Success/Error) ที่มุมจอ เพื่อให้ผู้ใช้รู้ผลลัพธ์โดยไม่ต้องเพ่งอ่าน JSON
- [ ] **Form Reset:** เมื่อส่งสำเร็จ ข้อมูลในทุกช่องต้องถูกล้างค่าให้ว่างเปล่าโดยอัตโนมัติ เพื่อพร้อมรับการกรอกครั้งถัดไป

C5: Error Handling (การรับมือเหตุการณ์ไม่คาดฝัน)

วิธีดูว่าผ่าน:

- [ ] **Try-Catch Block:** ในโค้ดต้องมีการใช้ `try { ... } catch (error) { ... }` เพื่อดักจับกรณีเน็ตหลุดหรือ Server พัง
- [ ] **Graceful Failure:** หากลองเปลี่ยน URL ของ API ให้ผิด แอปต้องไม่ค้าง (Crash) แต่ต้องแสดง Toast สีแดงแจ้งเตือนผู้ใช้แทน

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

Side Effects, Axios Data Fetching, API Optimization (Debouncing Logic), Skeleton Loading และ Conditional Rendering

---

Function 6.2 - Dynamic Product Searching & Display

---

1. จุดประสงค์

1. เพื่อฝึกการดึงข้อมูลจาก External API ด้วย Axios ภายในวงจรชีวิตของ Component (`useEffect`)
2. เพื่อประยุกต์ใช้เทคนิค **Debouncing** เพื่อลดการยิง API ที่ไม่จำเป็น (Optimization)
3. เพื่อจัดการสถานะการแสดงผลที่หลากหลาย (Loading, Success, Empty, Error)

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

---

2. โจทย์

พัฒนาหน้า **"Product Explorer"** ที่ดึงข้อมูลจาก `https://dummyjson.com/products` โดยมีเงื่อนไขดังนี้:

- **การดึงข้อมูล (Fetching):**
  - เมื่อโหลดหน้าจอครั้งแรก ให้ดึงข้อมูลสินค้าทั้งหมดมาแสดง
  - ข้อมูลที่แสดงต้องประกอบด้วย: รูปภาพสินค้า, ชื่อ, ราคา และหมวดหมู่ (Category)
- **ระบบค้นหา (Search Optimization):**
  - มีช่อง Search Input สำหรับค้นหาสินค้าตามชื่อ
  - **ห้ามยิง API ทุกครั้งที่พิมพ์:** ต้องใช้เทคนิค **Debouncing** (รอให้ผู้ใช้หยุดพิมพ์ 600ms ถึงจะยิง API ค้นหา)
- **การจัดการสถานะ (State Management):**
  - **Loading:** ระหว่างรอข้อมูล ต้องแสดง Skeleton Screen (แถบสีเทากะพริบ) หรือ Spinner
  - **Empty State:** หากค้นหาแล้วไม่พบสินค้า ให้แสดงข้อความ "No products found for '[Keyword]'"
  - **Error:** หาก API ล่ม ต้องแสดง UI แจ้งเตือนพร้อมปุ่ม 'Retry'

---

3. Coding

ต้องติดตั้ง `lodash.debounce` เพื่อช่วยจัดการเรื่องเวลา หรือเขียนฟังก์ชัน Debounce เอง

---

4. ขั้นตอนการทำ

1. **Dependency Setup:** ติดตั้ง `axios` และ `lodash.debounce`
2. **Debounce Implementation:** นิสิตต้องเขียน Logic ใน `useCallback` ให้ถูกต้อง เพื่อให้ `debouncedSearch` ทำงานเฉพาะเมื่อหยุดพิมพ์ครบตามเวลาที่กำหนดเท่านั้น
3. **UI Feedback:** ออกแบบส่วนแสดงผลระหว่างโหลด (Skeleton) ให้ดูเป็นมืออาชีพ (แนะนำให้ใช้ Tailwind CSS `animate-pulse`)

4. **Network Testing:** เปิด **Chrome DevTools > Network** แล้วเลือกความเร็วเป็น **"Slow 3G"** เพื่อทดสอบดูว่า Skeleton และ Loading Spinner ทำงานสัมพันธ์กับข้อมูลจริงหรือไม่
5. **API Verification:** สังเกตใน Network Tab ว่าเมื่อพิมพ์คำว่า "Macbook" รัวๆ ระบบจะยิง API ไปที่ Server **เพียงครั้งเดียว** หลังจากพิมพ์จบหรือไม่

---

5. คำถาม

1. หากไม่ใช้ `useCallback` ครอบฟังก์ชัน `debounce` จะเกิดปัญหาอะไรขึ้นเมื่อ Component มีการ Re-render?
2. ในกรณีที่ Network ช้ามาก และผู้ใช้พิมพ์ค้นหาคำที่ 1 แล้วตามด้วยคำที่ 2 ทันที แต่อาจเกิดปัญหา "Race Condition" (ผลลัพธ์ของคำแรกด้นมาถึงทีหลังคำที่สอง) นิสิตจะมีวิธีป้องกันปัญหานี้อย่างไร? (Hint: AbortController)

Checklist Function 6.2

C1: API Fetching & Rendering (การดึงและแสดงผล)

วิธีดูว่าผ่าน:

- [ ] **First Load:** เมื่อเปิดหน้าเว็บครั้งแรก (โดยยังไม่พิมพ์อะไร) ต้องมีรายการสินค้าโผล่ขึ้นมาทันทีอย่างน้อย 20-30 รายการ
- [ ] **Data Completeness:** ในแต่ละ Card สินค้า ต้องแสดงข้อมูลครบตามโจทย์ (รูปภาพ, ชื่อที่ไม่ออกนอกกรอบ, ราคาที่มีสกุลเงิน, และป้ายหมวดหมู่)
- [ ] **Key Property:** ตรวจสอบใน Console ของ Browser ต้อง **ไม่มี Error** แจ้งว่า `Warning: Each child in a list should have a unique "key" prop`

C2: Debouncing Logic (การหน่วงเวลา - สำคัญมาก)

วิธีดูว่าผ่าน:

- [ ] **Network Request Count:** เปิดหน้า **Inspect > Network** * ลองพิมพ์คำว่า `Samsung` แบบเร็วๆ รัวๆ
  - **ผลที่ต้องเห็น:** ใน Network Tab ต้องมีรายการวิ่งออกไป **เพียง 1 ครั้งเท่านั้น** หลังจากหยุดพิมพ์ไปแล้วประมาณครึ่งวินาที
  - **ถ้าผ่าน:** จะเห็น Request เดียวคือ `?q=Samsung`
  - **ถ้าตก:** จะเห็น Request ยิงออกไปทุกตัวอักษรที่พิมพ์ (`?q=S` , `?q=Sa` , `?q=Sam` ...)

C3: Skeleton Loading UX (ความลื่นไหลของหน้าจอ)

วิธีดูว่าผ่าน:

- [ ] **Slow Network Test:** ในหน้า Network ของ Browser เปลี่ยนสถานะจาก "No Throttling" เป็น **"Slow 3G"**
- [ ] **Visual Consistency:** เมื่อกดค้นหา หน้าจอต้องไม่ขาวว่างเปล่า แต่ต้องโชว์ **Skeleton Screen** (แถบสีเทาที่กะพริบหรือ Spinner) ขึ้นมาแทนที่ตำแหน่ง Card สินค้าทันที
- [ ] **Layout Stability:** เมื่อข้อมูลโหลดเสร็จ Skeleton ต้องหายไปและสินค้าต้องโผล่มาเสียบแทนที่ในตำแหน่งเดิมพอดี โดยที่หน้าจอไม่กระตุกหรือเลื่อน (No Layout Shift)

C4: Empty & Error States (การจัดการเคสพิเศษ)

วิธีดูว่าผ่าน:

- [ ] **No Results:** ลองพิมพ์คำค้นหามั่วๆ ที่ไม่มีในโลก (เช่น `xyz12345`)
  - **ผลที่ต้องเห็น:** หน้าจอต้องแสดงข้อความ **"No products found for 'xyz12345'"** อย่างชัดเจน ไม่ใช่โชว์หน้าว่างๆ
- [ ] **Error Handling:** ลองปิดเน็ตหรือแกล้งแก้ URL ในโค้ดให้ผิด
  - **ผลที่ต้องเห็น:** ต้องมีข้อความแจ้งเตือนสีแดง (Error Message) และ **ปุ่ม Retry** ที่กดแล้วจะพยายามดึงข้อมูลใหม่อีกครั้ง

C5: Component Optimization (การเขียนโค้ดที่ดี)

วิธีดูว่าผ่าน:

- [ ] **useCallback & useEffect:** ในโค้ดต้องมีการใช้ `useCallback` ครอบฟังก์ชัน `debounce` เพื่อป้องกันการสร้างฟังก์ชันใหม่พร่ำเพรื่อ

- [ ] **Cleanup Function:** ใน `useEffect` ต้องมีการเขียน `return () => debouncedSearch.cancel()` (ถ้าใช้ lodash) เพื่อล้างการทำงานค้างเก่าหาก User พิมพ์เร็วเกินไป

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

Context API for Multi-step Forms, Zod Schema, และ Persistence ใช้สถาปัตยกรรมแบบ Centralized State และการตรวจสอบข้อมูลแบบ Async Validation

---

Function 6.3 - Multi-Step Self-Correcting Registration Flow

---

1. จุดประสงค์

1. เพื่อฝึกการจัดการ State ของฟอร์มขนาดใหญ่ที่มีหลายขั้นตอนโดยใช้ **Context API**
2. เพื่อทำความเข้าใจการทำ **Async Validation** (การเช็คข้อมูลกับ Server ก่อนส่งจริง)
3. เพื่อฝึกการใช้ **Zod Schema** ในการทำ Conditional Validation (กฎที่เปลี่ยนตามเงื่อนไข)
4. เพื่อเรียนรู้การทำ **Data Persistence** ป้องกันข้อมูลหายเมื่อ Refresh หน้าจอ

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

---

2. โจทย์

พัฒนาหน้าสมัครสมาชิกที่มี 3 ขั้นตอน ดังนี้:

- **Step 1: Account Setup** (Email, Password, Username)
  - **เงื่อนไขพิเศษ:** ฟิลด์ **Username** ต้องมีการทำ **Async Validation** เพื่อเช็คว่า "admin", "root", หรือ "superuser" ห้ามใช้ (จำลองการเช็คกับ Database)
- **Step 2: Professional Profile** (Occupation, Company, GitHub/Portfolio URL)
  - **เงื่อนไขพิเศษ (Conditional):** หากผู้ใช้เลือก Occupation เป็น `Developer` ฟิลด์ **GitHub URL** จะต้อง "บังคับกรอก" และต้องเป็น Format URL ที่ถูกต้อง
- **Step 3: Summary & Confirm**
  - แสดงข้อมูลทั้งหมดที่กรอกมาเพื่อให้ตรวจสอบความถูกต้องก่อนกด Submit จริง

ข้อกำหนดทางเทคนิค:

1. **Persistence:** หากรีเฟรชหน้าจอ ข้อมูลที่กรอกไปแล้วต้องยังอยู่ (ใช้ `localStorage` ร่วมกับ `useEffect`)
2. **Navigation Guard:** ห้ามกดไป Step ถัดไป หาก Step ปัจจุบันยังมี Error หรือยังไม่ได้กรอกข้อมูลที่จำเป็น
3. **Architecture:** ต้องแยก Zod Schema ออกมาเป็นไฟล์หรือตัวแปรที่ชัดเจน ไม่เขียนปนกับ JSX

💡 คำแนะนำ (Pro-tips):

- การทำ Persistence ใน Multi-step ฟอร์มให้ใช้ `methods.reset(formData)` เมื่อข้อมูลถูกโหลดจาก LocalStorage กลับเข้ามาใน Context
- สำหรับการทำ Conditional Validation ใน Zod ให้ศึกษาคำสั่ง `.superRefine()` หรือการใช้ `.union()` เพื่อเช็คค่าของฟิลด์หนึ่งก่อนกำหนดกฎให้อีกฟิลด์หนึ่ง

Checklist Function 6.3

C1: Async Validation (การตรวจสอบ Username แบบหน่วงเวลา)

วิธีดูว่าผ่าน:

- [ ] **Debounced/Async Check:** เมื่อพิมพ์คำว่า `admin` หรือ `root` ลงไปในช่อง Username ระบบต้องไม่ด่าทันที แต่ต้องรอประมาณ 0.5 วินาที (Simulator) เพื่อเลียนแบบการวิ่งไปเช็คที่ Database
- [ ] **Error Message:** ต้องขึ้นข้อความเฉพาะเจาะจงว่า "Username นี้มีคนใช้ไปแล้ว" (ห้ามขึ้นแค่ว่า Invalid เฉยๆ)
- [ ] **Block Next Step:** ขณะที่กำลัง Async Check หรือหากเช็คแล้วไม่ผ่าน ปุ่ม "Next Step" จะต้องไม่ยอมให้ผ่านไปหน้า 2 เด็ดขาด

C2: Conditional Zod Validation (กฎตามเงื่อนไขอาชีพ)

วิธีดูว่าผ่าน:

- [ ] **The "Developer" Trigger:** ที่หน้า 2 ลองเลือก Occupation เป็น `Developer` แล้วปล่อยช่อง GitHub URL ว่างไว้ ➡️ **ต้องติด Error**
- [ ] **URL Format:** ในขณะที่เลือกเป็น `Developer` ให้ลองพิมพ์ข้อความมั่วๆ ที่ไม่ใช่ URL (เช่น `abc`) ➡️ **ต้องติด Error** แจ้งว่า Format ไม่ถูกต้อง
- [ ] **The "Other" Path:** ลองเปลี่ยน Occupation เป็นอาชีพอื่น เช่น `Designer` หรือ `Manager` แล้วปล่อยช่อง URL ว่างไว้ ➡️ **ต้องผ่านได้** (เพราะกฎบังคับเฉพาะ Developer)

C3: Data Persistence (หัวใจของการจัดการ State)

วิธีดูว่าผ่าน:

- [ ] **F5 Test:** กรอกข้อมูลหน้า 1 ให้ครบ แล้วกด **Refresh (F5)** ➡️ ข้อมูลในช่อง Email และ Password **ต้องยังอยู่** ไม่หายไป
- [ ] **Step Memory:** กรอกหน้า 1 เสร็จ กด Next ไปหน้า 2 แล้วกด **Refresh (F5)** ➡️ แอปต้องเปิดมาที่ **หน้า 2 ทันที** (ไม่ใช่กลับไปเริ่มหน้า 1 ใหม่)
- [ ] **LocalStorage Check:** เปิด Inspect > Application > Local Storage ➡️ ต้องเห็น Object ข้อมูลที่เรากรอกถูกเก็บไว้ในนั้นแบบ Real-time

C4: Context & Form Provider (โครงสร้างสถาปัตยกรรม)

วิธีดูว่าผ่าน:

- [ ] **Single Source of Truth:** เมื่อกด "Back" จากหน้า 2 มาหน้า 1 ข้อมูลเดิมต้องถูกเติมกลับเข้าไปใน Input (Auto-fill) โดยที่เราไม่ต้องกรอกใหม่
- [ ] **Clean Submission:** ในหน้าสุดท้าย (Step 3) ข้อมูลที่แสดงผลต้องเป็นก้อนเดียวกัน (Flattened Object) ที่รวมจากทั้งหน้า 1 และหน้า 2 มาแสดงผลได้ครบถ้วน

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

⚠️ ข้อควรระวัง (Common Pitfalls)

- **ลืม Reset:** บางคนโหลดข้อมูลจาก Local Storage มาแล้วแต่ลืมสั่ง `methods.reset(data)` ทำให้ข้อมูลใน Context เปลี่ยน แต่หน้าจอ (UI) ไม่เปลี่ยนตาม
- **Async Error:** หากกด Next รัวๆ ในขณะที่ Async Validation ยังทำงานไม่เสร็จ อาจทำให้เกิดบั๊กข้อมูลหลุดไปได้ จึงควรปิด (Disable) ปุ่ม Next ระหว่างที่สถานะเป็น `isValidating`

Performance UX และการจัดการ Reliability ของระบบเมืออินเทอร์เน็ตไม่เสถียร Axios Interceptors (Retry Logic), และ Error Rollback

---

Function 6.4 - The Optimistic Inventory Manager

---

1. จุดประสงค์

1. เพื่อฝึกการทำ **Optimistic UI Updates** เพื่อให้แอปพลิเคชันตอบสนองทันทีโดยไม่รอ Server
2. เพื่อเรียนรู้การทำ **State Rollback** เมื่อการเชื่อมต่อล้มเหลว
3. เพื่อใช้งาน **Axios Interceptors** ในการทำ **Automatic Retry Logic** (การลองส่งข้อมูลใหม่เมื่อเกิด Network Error)

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

---

2. โจทย์โดยละเอียด

ให้นิสิตพัฒนาหน้าจัดการสต็อกสินค้า (Inventory) โดยมีรายการสินค้าที่ดึงมาจาก API (จำลอง) และมีความสามารถดังนี้:

1. **Instant Delete:** เมื่อกดปุ่ม "ลบ" สินค้าต้องหายไปจากหน้าจอทันที (Optimistic) โดยไม่ต้องรอ API ตอบกลับ
2. **Instant Toggle:** เมื่อกดปุ่ม "In Stock / Out of Stock" สถานะต้องเปลี่ยนทันที
3. **Advanced Retry Logic:** หากการส่งข้อมูลล้มเหลวเนื่องจากปัญหา Network ให้ Axios ลองใหม่ (Retry) อัตโนมัติสูงสุด 3 ครั้ง
4. **Graceful Rollback:** หากลองครบ 3 ครั้งแล้วยังล้มเหลว (เช่น Server พัง) ระบบต้อง "ดึด" สินค้าที่ลบไปแล้วกลับมาที่หน้าจอ และแจ้งเตือน User ว่า "การซิงค์ล้มเหลว"

เงื่อนไขความยาก:

- ห้ามใช้ Library สำเร็จรูปในการทำ Optimistic (เช่น React Query) ต้องเขียน Logic จัดการ State เอง
- ต้องตั้งค่า Axios Instance ให้รองรับ **Interceptors** เพื่อดักจับ Error และสั่ง Retry

---

🛠 ตาราง Tester (ไม่ใช่ Checklist)

กรณีทดสอบ	การกระทำ	สิ่งที่ต้องเห็น	ผ่าน/ไม่ผ่าน
Normal Case	กดลบตอนเน็ตดี	สินค้าหายไปทันทีและไม่เด้งกลับมา	[ ]
Network Failure	ปิดเน็ตแล้วกดลบ	สินค้าหายไป -> แถบ Retry วิ่ง -> สินค้าเด้งกลับมา	[ ]
Rapid Actions	กด Toggle รัวๆ	UI ต้องเปลี่ยนตามนิ้วทันที และ Final State ต้องตรงกับที่กดล่าสุด	[ ]

คำแนะนำเพิ่มเติม: การทำ Rollback ในฟอร์มที่มีข้อมูลซับซ้อน นิสิตต้องระวังเรื่อง "Race Condition" (การที่ Request ใหม่แซงอันเก่า) แนะนำให้ใช้ `previousItems` ที่ดักจับ State ณ เสี้ยววินาทีก่อนสั่ง Update

การตรวจงานจะไม่ใช่แค่ดูว่า "ปุ่มกดติดไหม" แต่ต้องดูว่า "ระบบจัดการกับความผิดพลาดได้เนียนแค่ไหน"

Checklist Function 6.4

C1: Instant UX (การตอบสนองแบบทันที)

วิธีดูว่าผ่าน:

- [ ] **The "No Spinner" Test:** เมื่อคุณกดปุ่ม **Delete** หรือ **Toggle** สินค้าต้องหายไปหรือเปลี่ยนสถานะ **ทันที (ภายในเวลาไม่เกิน 0.1 วินาที)** * [ ] **Behavior:** หากกดแล้วเห็น "วงกลมหมุนๆ (Loading Spinner)" ก่อนที่ข้อมูลจะเปลี่ยน ➡️ **สอบตก** เพราะนั่นคือการอัปเดตแบบปกติ ไม่ใช่ Optimistic
- [ ] **Smooth Transition:** การหายไปของสินค้าควรจะลื่นไหล ไม่มีการกระตุกของ Layout (Layout Shift)

C2: Network Retry Logic (การพยายามใหม่เมื่อพัง)

วิธีดูว่าผ่าน:

- [ ] **Inspect Network Tab:** เปิด F12 > Network และจำลองการ Error (เช่น การแก้ URL ในโค้ดให้ผิด หรือใช้ฟีเจอร์ "Offline" ในเบราว์เซอร์)
- [ ] **Retry Count:** เมื่อกดลบ คุณต้องเห็น Request เดิมในหน้า Network วิ่งออกไป **ซ้ำกัน 3 ครั้ง** (ตามที่ตั้งค่าไว้ใน Interceptor) โดยแต่ละครั้งควรมีช่วงเวลาห่างกันเล็กน้อย (Delay)
- [ ] **Console Log:** หากเขียน `console.log` ไว้ใน Interceptor ต้องเห็นข้อความ "Retrying... (1)", "Retrying... (2)" ปรากฏขึ้นมา

C3: State Rollback (การดีดกลับเมื่อซิงค์ไม่สำเร็จ)

วิธีดูว่าผ่าน:

- [ ] **The "Reappearance" Test:** หลังจากที่ระบบพยายาม Retry จนครบ 3 ครั้งแล้ว (และยังพังอยู่) สินค้าที่หายไปจากหน้าจอ **ต้องเด้งกลับมาปรากฏที่เดิม** โดยอัตโนมัติ
- [ ] **Data Integrity:** ข้อมูลที่เด้งกลับมาต้องมีค่าเหมือนเดิมทุกประการ (เช่น ชื่อเดิม, สถานะ Stock เดิม) ไม่มีการผิดเพี้ยน
- [ ] **No Ghost State:** ต้องไม่มีกรณีที่ "สินค้าหายไปแล้วหายเลย" ทั้งที่ API แจ้งว่า Error เพราะจะทำให้ข้อมูลหน้าบ้านกับหลังบ้านไม่ตรงกัน

C4: User Notification (การสื่อสารกับผู้ใช้)

วิธีดูว่าผ่าน:

- [ ] **Optimistic Toast:** เมื่อเริ่มกดปุ่ม ควรมี Toast แจ้งว่า "ลบสินค้าแล้ว (กำลังซิงค์...)" เพื่อบอกผู้ใช้ว่าเราทำตามคำสั่งแล้วนะ แต่ยังรอการยืนยันจากเซิร์ฟเวอร์อยู่
- [ ] **Error Toast:** เมื่อ Rollback เกิดขึ้น ต้องมี Toast สีแดงแจ้งเตือนชัดเจนว่า "ไม่สามารถดำเนินการได้ ระบบคืนค่าเดิมให้แล้ว" เพื่อให้ผู้ใช้ไม่สับสนว่าทำไมสินค้าถึงเด้งกลับมา

C5: Code Robustness (ความแข็งแรงของโค้ด)

วิธีดูว่าผ่าน:

- [ ] **Closure State:** ในโค้ดต้องมีการใช้ `const previousItems = [...items]` เพื่อ Backup ข้อมูลไว้ **ก่อน** สั่ง `setItems` เสมอ
- [ ] **Error Propagation:** ใน Interceptor ต้องมั่นใจว่าสุดท้ายแล้วมีการส่ง `return Promise.reject(error)` ออกไป เพื่อให้บล็อก `catch` ในตัว Component ทำงานได้

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

Dynamic Rendering, Recursive Components, และ Dynamic Zod Schema Generation ไม่เขียนฟอร์ม Hardcode แต่สร้างระบบที่อ่านโครงสร้างจาก JSON และสร้าง UI พร้อม Validation ออกมาอัตโนมัติ

Function 6.5 - Dynamic Form Engine (Schema-to-UI)

---

1. จุดประสงค์

1. เพื่อฝึกการสร้าง Component แบบ **Recursive** เพื่อรองรับโครงสร้างข้อมูลที่ซับซ้อน (Nested Data)
2. เพื่อเรียนรู้การแปลง JSON Metadata ให้กลายเป็น **Dynamic Validation Schema**
3. เพื่อจัดการสถานะฟอร์มที่จำนวนและประเภทของ Input เปลี่ยนแปลงตาม Data ที่ได้รับจาก API

---

2. โจทย์โดยละเอียด

ให้นิสิตพัฒนา **"Form Engine"** ที่รับค่า JSON จาก Backend แล้วสร้างฟอร์มออกมาให้ทำงานได้จริง โดยมีเงื่อนไขดังนี้:

1. **Schema Interpretation:** ระบบต้องอ่านไฟล์ JSON ที่มีนิยามของ `type` (text, select, checkbox), `name`, `label`, และ `validation_rules`
2. **Nested Object Support:** หาก JSON มีโครงสร้างเป็น Object ซ้อน Object ระบบต้อง Render ออกมาเป็นส่วนย่อย (Sub-section) ได้ถูกต้อง
3. **Dynamic Zod Generation:** ห้ามเขียนกฎ Validation ตายตัวในโค้ด แต่ต้องเขียนฟังก์ชันที่วนลูปอ่าน `validation_rules` ใน JSON แล้วสร้าง `z.object()` ออกมาแบบ Runtime
4. **Conditional Display:** หากฟิลด์ใดมีพร็อพเพอร์ตี้ `show_if` (เช่น `"show_if": { "field": "role", "equals": "admin" }`) ฟิลด์นั้นจะปรากฏขึ้นเฉพาะเมื่อเงื่อนไขเป็นจริงเท่านั้น

🛠 ตาราง Tester (ไม่ใช่ Checklist)

สถานการณ์	การกระทำ	ผลลัพธ์ที่ถูกต้อง	ผ่าน
Schema Update	เพิ่มฟิลด์ "Age" ใน JSON	ฟิลด์ Age ปรากฏบน UI ทันที	[ ]
Logic Check	เลือก Role เป็น `guest`	ฟิลด์ Admin Secret ต้องถูกลบออกจาก DOM	[ ]
Validation	ทิ้งช่อง Required ให้ว่าง	ต้องขึ้น Error ตามที่ระบุใน JSON rules	[ ]
Submission	กรอกข้อมูลครบถ้วน	ได้ก้อน Object JSON ที่ตรงตามโครงสร้างที่ต้องการ	[ ]

คำแนะนำพิเศษ: Function นี้จะทดสอบทักษะการออกแบบ **Abstraction** ของคุณ หากคุณทำสำเร็จ คุณจะสามารถสร้างระบบหลังบ้าน (Admin Panel) ที่ทรงพลังมาก ซึ่งสามารถเปลี่ยนฟอร์มได้เพียงแค่การแก้ไข Configuration บน Database

การตรวจสอบจะไม่ได้ดูที่ความสวยงาม แต่ดูที่ **"ความฉลาดของ Engine"** ว่ามันอ่านคำสั่งจากข้อมูล (Data-driven) ได้แม่นยำแค่ไหน

Checklist Function 6.5

C1: Dynamic Field Rendering (การสร้าง UI จากข้อมูล)

วิธีดูว่าผ่าน:

- [ ] **The "No Code Change" Test:** ให้นักศึกษาลองเพิ่ม Object ฟิลด์ใหม่เข้าไปในไฟล์ JSON (เช่น เพิ่มฟิลด์ `phoneNumber` ประเภท `text`)
- [ ] **Behavior:** เมื่อกด Save ไฟล์ JSON ปุ๊บ หน้าเว็บต้องแสดงช่องกรอกเบอร์โทรศัพท์ขึ้นมาทันที **โดยที่นักศึกษาห้ามพิมพ์คำว่า `<input>` หรือ `phoneNumber` ลงในไฟล์ `.jsx` หรือ `.tsx` แม้แต่คำเดียว**
- [ ] **Type Correctness:** หากใน JSON ระบุ `type: "select"` ต้องขึ้นเป็น Dropdown หากระบุ `type: "text"` ต้องขึ้นเป็นช่องกรอกปกติ

C2: Runtime Validation Generation (การสร้างกฎตรวจสอบสดๆ)

วิธีดูว่าผ่าน:

- [ ] **The "Logic Mapping" Test:** ใน JSON ลองเปลี่ยนค่า `"min": 5` เป็น `"min": 15`
- [ ] **Behavior:** เมื่อพิมพ์ในหน้าเว็บ 10 ตัวอักษร ระบบต้องขึ้น Error ทันที (เพราะกฎใหม่คือ 15)
- [ ] **Schema Integrity:** หากนักศึกษาเขียนโค้ดได้ถูกต้อง ตัวแปร `dynamicSchema` ใน `useMemo` จะต้องเปลี่ยนโครงสร้างไปตาม JSON ทุกครั้งที่มีการแก้ไข (ตรวจสอบได้โดยการ `console.log(dynamicSchema)` ออกมาดู)

C3: Conditional Display Logic (การซ่อน/แสดงตามเงื่อนไข)

วิธีดูว่าผ่าน:

- [ ] **The "Role" Switch:** ลองเลือก Role เป็น `guest` ➡️ ฟิลด์ `admin_code` ต้องหายไปจากหน้าจอ (ถูกลบออกจาก DOM)
- [ ] **The "Admin" Switch:** เมื่อเปลี่ยนกลับเป็น `admin` ➡️ ฟิลด์ `admin_code` ต้องโผล่กลับมาทันที
- [ ] **Validation Sync:** เมื่อฟิลด์ `admin_code` แสดงขึ้นมา มันต้องติดสถานะ `required` ตามที่ JSON ระบุไว้ด้วย (ถ้าหายไปแล้ว กฎ validation ของมันต้องหายไปด้วยเพื่อไม่ให้ขัดขวางการ Submit)

C4: Payload Construction (โครงสร้างข้อมูลที่ส่งออก)

วิธีดูว่าผ่าน:

- [ ] **The "Final JSON" Test:** เมื่อกรอกข้อมูลครบทุกช่องแล้วกด Submit
- [ ] **Behavior:** ใน `console.log` ข้อมูลที่ได้รับ (Payload) ต้องมี Key ตรงตามชื่อ `name` ที่ระบุไว้ใน JSON ทุกประการ
- [ ] **Hierarchy:** หากโจทย์ระบุให้รองรับ Nested (ซ้อนกัน) ข้อมูลที่ออกมาต้องมีโครงสร้างเป็น Object ซ้อน Object ตามโครงสร้างต้นฉบับ ไม่ใช่กลายเป็นก้อนแบนๆ (Flat object) ทั้งหมด

C5: Component Recursion (กรณีมีฟิลด์ซ้อนกัน)

วิธีดูว่าผ่าน:

- [ ] **Self-Calling Component:** หากใน JSON มีฟิลด์ประเภท `group` หรือ `section` ที่มี `fields` ข้างในซ้อนลงไปอีก
- [ ] **Behavior:** ตัว Component `DynamicField` ต้องสามารถเรียกตัวเอง (Recursive) เพื่อ Render ฟอร์มย่อยข้างในออกมาได้โดยไม่เกิด Infinite Loop หรือ Error

---

💡 **ข้อแนะนำ**

จุดที่ยากที่สุดคือการใช้ **`useWatch`** ของ React Hook Form เพื่อทำเงื่อนไขการซ่อน/แสดง และการใช้ **`zodResolver`** กับ Schema ที่สร้างขึ้นแบบ Dynamic หากทำข้อนี้ผ่าน สามารถนำ Logic นี้ไปสร้างระบบ **CMS** หรือ **E-commerce Admin** ที่ปรับแต่งหน้าบ้านได้จากระบบจัดการข้อมูล (Configuration) ได้